Comparativa de rendimiento: Pandas vs Polars vs PySpark

Dataset: MovieLens 20M — 20,000,263 filas × 6 columnas

Mediciones reales sobre hardware: 32 cores, 15 GB RAM, WSL2 (Linux 5.15 / Windows)
Dataset generado uniendo `rating.csv` + `movie.csv` → `movielens_superdataset.csv`

---

1. Tabla de resultados

Operación	Pandas	Polars Eager	Polars Lazy	PySpark (6g)
Carga CSV	14.438 s	**1.124 s**	— [1]	5.64 s [2]
Filtro Action ≥ 4	1.145 s	**0.108 s**	0.817 s [3]	0.47 s
Agregación por usuario	0.337 s	0.284 s	0.575 s [3]	**0.60 s**
Ordenación por rating	0.547 s	**0.303 s**	0.771 s [3]	0.67 s
Conversión timestamp	15.030 s	**0.080 s**	0.561 s [3]	0.08 s
TOTAL	**31.497 s**	**1.899 s**	**2.724 s** [3]	**6.86 s** [2]

[1] `scan_csv` no carga el archivo — el tiempo está repartido en cada operación
[2] Incluye overhead de inicio de JVM + SparkSession (~1–2 s amortizado) y cache inicial
[3] Cada operación Lazy incluye el escaneo completo del CSV desde disco (sin cache entre operaciones)

---

2. Análisis operación por operación

2.1 Carga del CSV

Herramienta	Tiempo	Por qué
Pandas	14.4 s	Lee fila a fila en Python, infiere tipos en Python, construye un DataFrame en memoria con copia de datos
Polars Eager	**1.1 s**	Motor en Rust con SIMD y lectura paralela por chunks; inferencia de tipos nativa sin overhead de Python
PySpark	5.6 s	Spark lee en paralelo (32 particiones), pero añade overhead de serialización JVM + inferSchema (2 pasadas)

Clave: Polars es **13× más rápido que Pandas** en carga porque su parser CSV está implementado en Rust con paralelismo a nivel de núcleo y sin el GIL de Python.

2.2 Filtro (genres contiene "Action" AND rating ≥ 4)

Herramienta	Tiempo	Por qué
Pandas	1.145 s	`str.contains` opera en Python sobre cada elemento del Series; el AND de máscaras es otro pase
Polars Eager	**0.108 s**	Expresión compilada a Rust; el motor evalúa ambas condiciones en un solo pase paralelo sobre los chunks
Polars Lazy	0.817 s	Incluye releer el CSV desde disco; el filtro en sí es igual de rápido pero no hay cache previo

Herramienta	Tiempo	Por qué
PySpark	0.47 s	Dato ya cacheado en RAM distribuida; Catalyst aplica predicate pushdown y el filtro es rápido

Clave: Con datos ya en memoria (Eager/PySpark cached), el filtro es muy rápido en todos. La diferencia Pandas vs Polars viene del overhead de Python en operaciones de string.

2.3 Agregación por usuario (mean + std de rating)

Herramienta	Tiempo	Por qué
Pandas	0.337 s	`groupby` + `agg` usa NumPy internamente — eficiente para operaciones numéricas simples
Polars Eager	**0.284 s**	Similarmente rápido; motor Rust con hash aggregation paralela
PySpark	0.60 s	También eficiente, pero el shuffle de datos entre particiones añade latencia incluso en local

Clave: Para agregaciones numéricas puras Pandas y Polars son comparables porque Pandas delega a NumPy (Fortran/C). La ventaja de Polars se nota más en strings y pipelines complejos.

2.4 Conversión de timestamp (string → datetime)

Herramienta	Tiempo	Por qué
Pandas	**15.030 s**	`pd.to_datetime` con `format=` itera en Python elemento a elemento sobre 20M strings — el peor caso de Pandas
Polars Eager	**0.080 s**	Parser de fechas compilado en Rust; procesa los 20M strings en paralelo
PySpark	0.08 s	`to_timestamp` en la JVM + Catalyst — también muy rápido en operaciones de columna completa

Clave: Este es el caso donde Pandas falla más dramáticamente. Operaciones de string/fecha son el punto débil de Pandas porque escapan a NumPy y vuelven a Python puro. Polars es **188× más rápido** en esta operación concreta.

---

3. Por qué cada herramienta es más o menos rápida

Pandas — ¿Por qué es lento en algunos casos?

```
Arquitectura: Python → NumPy (C/Fortran)
```

- **GIL de Python:** no puede paralelizar operaciones a nivel de núcleo
- **Columnas de objetos:** strings se almacenan como Python `object` (punteros a objetos Python), no como buffers contiguos en memoria → mal rendimiento en cache de CPU
- **Operaciones de string:** `str.contains`, `str.to_datetime` son loops Python disfrazados de métodos
- **Sin lazy evaluation:** cada operación se ejecuta inmediatamente aunque el resultado no sea necesario todavía

Polars — ¿Por qué es rápido?

`Arquitectura: Python API → Rust (SIMD + multithreading nativo)`

- **Motor en Rust:** sin GIL, sin overhead de intérprete Python en el camino caliente
- **Formato Arrow columnar:** datos en buffers contiguos en memoria, óptimo para el cache de CPU y SIMD
- **Paralelismo automático:** divide columnas en chunks y los procesa en paralelo sin configuración
- **LazyFrame:** el optimizador puede aplicar:
  - *Predicate pushdown*: aplica filtros antes de cargar datos completos
  - *Projection pushdown*: lee solo las columnas necesarias
  - *scan_csv*: no carga el archivo hasta `collect()`, puede optimizar el pipeline completo

PySpark — ¿Por qué tiene overhead pero escala mejor?

`Arquitectura: Python API → JVM (Scala) → ejecutores distribuidos`

- **Overhead de inicio:** la JVM tarda en arrancar; `SparkSession` + configuración de memoria = ~2-3s fijos
- **Catalyst Optimizer:** analiza el DAG completo de transformaciones y genera un plan físico optimizado (similar al LazyFrame de Polars pero más potente)
- **Shuffle:** operaciones como `groupBy` requieren redistribuir datos entre particiones → costoso en local, eficiente en clúster con más nodos
- **Ventaja real:** el mismo código que corre en local sobre 20M filas funciona en un clúster sobre **20 mil millones de filas** sin cambiar nada

---

4. Lazy vs Eager en Polars

En este benchmark, **Polars Eager ganó a Polars Lazy** en el total porque:

1. Con Eager, el CSV se carga **una vez** en RAM y todas las operaciones trabajan sobre esa copia en memoria
2. Con Lazy y `scan_csv`, **cada operación relee el archivo desde disco** (no hay cache entre llamadas)

Lazy sería más rápido que Eager cuando:

- Se encadenan múltiples operaciones en **una sola llamada** (un `collect()` al final)
- El dataset es **más grande que la RAM** — Lazy puede procesar en streaming sin cargar todo
- Se combinan filtros + proyecciones — el optimizador elimina trabajo antes de leer

```python
# Lazy CORRECTO (un solo collect — el optimizador trabaja sobre el plan completo)
resultado = (
    pl.scan_csv(DATA_PATH)
    .filter(pl.col("genres").str.contains("Action") & (pl.col("rating") >= 4))
    .group_by("title")
    .agg(pl.col("rating").mean().alias("avg_rating"))
    .sort("avg_rating", descending=True)
    .collect()   # ← un solo collect al final
)
```

---

5. Casos de uso recomendados

Escenario	Herramienta recomendada	Razón
Análisis exploratorio, notebook rápido	**Pandas**	Ecosistema maduro, más documentación, integración con sklearn/matplotlib/seaborn
Dataset < 10 GB en un solo nodo	**Polars Eager**	Mucho más rápido que Pandas, API expresiva, no necesita configuración
Pipeline complejo con muchos pasos encadenados	**Polars Lazy**	El optimizador elimina pasos redundantes; útil cuando el dataset no cabe entero en RAM
Dataset > RAM disponible (un solo nodo)	**Polars Lazy** (`scan_csv`)	Procesa en streaming sin necesidad de clúster
Dataset distribuido (10+ GB, múltiples máquinas)	**PySpark**	Escala horizontalmente; el mismo código funciona en clúster con Databricks/EMR
ETL de producción en plataforma cloud	**PySpark**	Integración nativa con HDFS, Delta Lake, Parquet, Hive; tolerancia a fallos
Prototipo rápido de ML	**Pandas / Polars**	Menos overhead; mejor integración con scikit-learn, PyTorch, etc.
Feature engineering a escala	**PySpark MLlib**	Puede entrenar modelos sobre datasets que no caben en un nodo

---

6. Resumen ejecutivo

```
Pandas:       simple, maduro, lento en strings y datasets grandes
Polars Eager: 13× más rápido que Pandas en carga, 188× en timestamps — mejor opción en un
 ↪  solo nodo
Polars Lazy:  igual de rápido si se encadena bien; ventaja real con datos > RAM
PySpark:      overhead fijo pero escala ilimitado — la única opción para Big Data real
 ↪  distribuido
```

La elección no es "cuál es mejor" sino **cuál es adecuada para el tamaño y el entorno del problema**:

- Si caben en RAM y trabajas solo → **Polars**
- Si no caben en RAM o necesitas un clúster → **PySpark**
- Si el equipo ya conoce Pandas y el dataset es pequeño → **Pandas** es perfectamente válido